Runtime Reference Safety via Access Path Invalidation

Vineeth Kashyap*[1], Wolfgang Grieskamp[1], and Teng Zhang[1]

Aptos Labs, Palo Alto, USA
vineeth@aptoslabs.com

Abstract. Move's bytecode verifier statically enforces a small set of reference safety rules: references should not dangle, reads through an immutable reference should not observe a value that has been changed through another handle, and mutable references at function call boundaries should not alias. In several contexts the static check is unavailable, untrusted, or unnecessarily conservative: bytecode that bypasses the verifier (in fuzzing or mutated-bytecode harnesses, behind dynamic dispatch), operations whose body lives outside the bytecode language (native code), and programs the verifier rejects even though the executions they actually take would not violate any rule. We describe a runtime discipline that enforces Move's reference-safety rules during execution. The core data structure is the *Access Path Tree* (APT): a forest, built lazily during execution, whose nodes record the access paths along which each live reference was derived. A single invalidation rule poisons references whose paths have been disturbed by destructive events; the rule covers destructive writes, moves, freezes, variant mutations, calls, returns, and opaque effects uniformly, and call boundaries additionally lock subtrees so that mutable arguments do not overlap. The discipline is path-sensitive—it runs against the actual execution trace, rather than against a join over branches—and is designed to accept every trace the static check would. We describe the model, sketch its small-step rules, and discuss its relation to per-location dynamic alias models such as Stacked Borrows and Tree Borrows.

Keywords: Runtime verification · Reference safety · Aliasing · Access paths · Move.

1 Introduction

Move is a bytecode language with first-class references over local and globally-stored values. Each function's bytecode passes through a static reference-safety verifier, which approves the function only if, on every reachable execution path, references stay in scope, do not violate type safety through reads or writes, do not overlap as mutable arguments at call boundaries, and do not allow an immutable view to observe a value rewritten through another handle. The verifier rejects bytecode that violates these rules.

* Corresponding author.

We are interested in enforcing the same rules *at execution time*. There are several reasons one would want this:

- Fuzzing and mutated-bytecode harnesses deliberately exercise inputs the verifier would reject, to find bugs in everything downstream.
- Dynamic dispatch and native code escape static analysis: the verifier cannot see what happens through a dispatched function pointer or inside a Rust-implemented native, but the reference rules still need to hold across those boundaries.
- The static check has to be sound, so it sometimes rejects programs that would not actually violate any rule on the executions they take. A runtime check sees the executions and can be more precise.

The following Move fragment illustrates the kind of subtle case at stake:

```
enum E { V1(T1), V2(T2) }
let v: E = ...;
let inner: &mut T1 = match &mut v { V1(p) => p };
... // some path mutates v to be V2
*inner // unsafe
```

The reference `inner` is in scope. Its lifetime has not ended. Its declared type is unchanged. But the program has switched `v` from `V1` to `V2`, so the substructure that `inner` pointed into is no longer there. Dereferencing `inner` now reads memory that has been recycled into a value of an unrelated type.

A sound static analysis catches this only conservatively: it does not know which branch will run, so it must assume the worst. By the time `*inner` executes, the runtime has already observed which branch ran, which mutation occurred, and which reference was derived from which. With the right slice of that information recorded, we can answer "is this reference still safe to use?" precisely. The challenge is to find a representation that is small enough to be cheap to maintain, and uniform enough to cover the various events that can affect a reference.

This paper. We organize live references by the path of borrows that produced them, using a hierarchical structure we call an *Access Path Tree* (APT). Each live reference is attached to a node of some APT, with edges labelled by field projections, variant projections, and a single label for vector indices. A single rule decides when a reference becomes unusable: when an event destructively changes the substructure reachable through some node, every reference attached to a related node is marked *poisoned* for the rest of the execution. Destructive writes, moves and overwrites, freezes, variant mutations, calls, returns, and opaque effects all fit this pattern; what varies between them is which set of related nodes is affected and which mutability filter applies. Calls additionally lock subtrees so that two mutable arguments cannot overlap.

Contributions.

- A runtime discipline for Move's reference-safety rules, organized around access path trees and one invalidation rule (Sects. 3, 4).
- A small-step presentation in which the various reference-affecting events are instances of the same rule (Sect. 4).
- A discussion of what the runtime trace buys over a join-based static check (Sect. 5) and how the discipline relates to per-location dynamic alias models such as Stacked Borrows and Tree Borrows [2,6] (Sect. 7).

2 Motivating Examples

The enum example in Sect. 1 is the headline case: mutation to a parent value invalidates a reference still in scope below it. Two further examples make the design constraints concrete.

Order matters. Here are two fragments that differ only in the order of their last two statements:

```
let x = 0; let r2 = &mut x; let r1 = &mut x; // (a)
*r2 = 5; *r1
let x = 0; let r2 = &mut x; let r1 = &mut x; // (b)
*r1 = 5; *r2
```

Both fragments take two mutable references to `x`. Fragment (a) writes through one of them and reads through the other; fragment (b) does the same with the roles swapped. A static borrow check sees the same set of borrows in scope at the same program points and has to reject both. The runtime sees the actual sequence of operations and can be more precise: the read after the write is safe in either fragment, because the read does not modify state and the earlier write through the other handle has not been challenged by a later write. A small variant—`let r1 = r2;`—makes both handles copies of one reference, and the question becomes uninteresting.

Two mutable arguments must not overlap. A function takes two mutable references. The caller must make sure they refer to disjoint pieces of state, since otherwise the callee cannot reason about either argument locally. Passing `&mut x.f` and `&mut x.g` is fine—they touch different fields. Passing `&mut x` and `&mut x.g` is not—the second is reachable through the first. The runtime check is substructural: it has to ask, for any two reference arguments, whether one is reachable from the other through field projections, indexing, and so on, along the actual paths the program took.

The three examples look different on the surface: variant mutation, mutable aliasing, and an overlap at a call boundary. But in every case, one operation invalidates the path through which another reference would read or write state. That observation is what motivates the model. The next section describes the data structure that records those paths; Sect. 4 describes the single rule that decides which references the next destructive event invalidates.

3 Access Path Trees

An access path describes how the program reached a particular substructure of Move's state: it starts at some named entry point—a local of the current frame, a globally-stored resource, or a reference parameter passed in by the caller—and follows a sequence of field projections, variant projections, and indexings. Every borrow in the program produces a reference that can be described by its access path. The runtime records these paths explicitly: it groups them into a forest of trees, one per entry point, and attaches each live reference to the node at the end of its path.

A reference is *valid* when the path it represents still leads to a reachable, type-compatible substructure of the current state. The runtime approximates this by marking references invalid when destructive events happen along their paths; the rules in Sect. 4 spell out which events invalidate which references.

Derivation. A reference ρ_2 is *derived from* ρ_1 when its access path is built by extending the path of ρ_1 through one of the language's borrowing operations: field projection, variant projection, indexing, freezing, or being passed across a function call as a reference parameter. In the access path tree this comes out simply: ρ_2 is derived from ρ_1 exactly when its node lies in the subtree below ρ_1's. The full derivation order is the reflexive-transitive closure.

Roots and edges. Each tree's root is one of the named entry points mentioned above. There are three kinds:

- $\mathrm{loc}(i)$, the i-th local of the current frame;
- $\mathrm{glb}(\tau)$, the global resource of type τ (we abstract addresses);
- $\mathrm{par}(i)$, the abstract value behind the i-th reference parameter passed in by the caller.

Edges carry labels: a field offset for record fields, a variant field for variant projections, and a single distinguished label 0 for vector elements. The vector label is an over-approximation: rather than track each index, we collapse all of them. This costs precision on element-wise borrows but keeps the trees small.

Trees are built lazily. The runtime never materializes a node until the program actually borrows through it.

State. A runtime state is a tuple $\sigma = (T, R, S, F)$.

- $T : \mathrm{Root} \rightharpoonup \mathrm{APT}$ maps each root to its tree. The map is partial: roots that have not been borrowed have no tree.
- R assigns each live reference identifier a triple (μ, π, n). Here $\mu \in \{\mathsf{mut}, \mathsf{imm}\}$ is the mutability, $\pi \in \{\checkmark, \dagger\}$ is the validity flag (we call $\dagger$ *poisoned* for short), and $n = \langle \mathrm{root}, \mathrm{id} \rangle$ is the reference's *qualified node*—the tree node it is attached to.
- S is a shadow stack mirroring the machine's operand stack. Each entry is either a non-reference token or a reference identifier.

– F is the per-frame stack. Each frame carries its own T and R, a vector of shadow locals, and a map from this frame's reference-parameter indices back to qualified nodes in the caller, which is what return transport uses.

Aliasing as a tree relation. Two references alias when one's node is an ancestor or descendant of the other's. The runtime decides this by walking parent pointers, in time linear in the depth of the tree. Each node also holds the set of references attached to it, so an invalidation walk only has to visit the affected subtree—it does not have to scan the whole reference table.

4 Invalidation Rules

We give the rules as a small-step relation $\sigma \to \sigma'$ indexed by the bytecode instruction being executed. Rules for operations that do not touch the shadow state—constants, arithmetic, stack reshuffles—are omitted.

Every non-trivial rule has the same shape. An event destructively changes the substructure reachable through some node n, and the runtime walks the tree from n to poison every reference attached to nodes related to n. Different events pick different sets of related nodes (self, ancestors, descendants, or unions) and different mutability filters. We write invalidate(N, ϕ) for the operation that poisons every reference attached to some node in N, filtered by $\phi \in \{\mathsf{mut}, \mathsf{imm}, \mathsf{all}\}$. A poisoned reference cannot be borrowed from, read, written, frozen, or passed as a call argument; any such attempt is a runtime safety violation.

Borrow, read, freeze. Borrowing a local i creates a fresh reference attached to $\mathrm{loc}(i)$, materializing $T(\mathrm{loc}(i))$ if it does not yet exist. A field borrow with label ℓ from a reference ρ at node n consumes ρ and creates a fresh reference at the child $n.\ell$. `ReadRef` pops the top reference after a validity check; the node is unchanged. `FreezeRef` on a live (mut, n) reference consumes it and produces a fresh (imm, n) reference at the same node—an immutable re-view of the same path. None of these events invalidates anything.

Destructive write. `WriteRef` through a mutable reference at n overwrites the value at n, which affects both its ancestors (whose contents have changed) and its descendants (which may have been replaced). The rule is:

$$\mathrm{invalidate}\big(\{n\} \cup \mathcal{D}(n) \cup \mathcal{A}(n),\ \mathsf{imm}\big), \qquad \mathrm{invalidate}\big(\mathcal{D}(n),\ \mathsf{mut}\big).$$

The first line poisons every immutable view of n, of any ancestor, and of any descendant. The second poisons every mutable view of a descendant. Mutable views of n itself are deliberately spared: a program that consumes one mutable view and immediately reborrows from n stays accepted, which matches static borrow checking.

Move, overwrite, variant mutation. Moving or storing over local i replaces the value at $\mathrm{loc}(i)$ outright, so we invalidate every reference reaching through it, regardless of mutability: $\mathrm{invalidate}(\{r_i\} \cup \mathcal{D}(r_i),\ \mathsf{all})$. Variant mutation is the same rule applied at the discriminator: references derived from the prior variant's projection sit in descendant nodes, and the change to the discriminator poisons them. The enum example from Sect. 1 falls out of this case.

Calls. A call $f(\overline{a})$ with reference arguments $a_{i_1}, \ldots, a_{i_k}$, each at node n_j with mutability μ_j, proceeds in four steps.

1. Check each ρ_j is live. Then lock the subtree at n_j: exclusively if $\mu_j = \mathsf{mut}$, shared otherwise. An exclusive lock conflicts with any other lock on a related node; a shared lock conflicts only with an exclusive one. Any conflict is a violation. This is what catches overlapping mutable arguments from Sect. 2.
2. For each mutable argument, apply the destructive-write rule at n_j. The callee may write through the reference, so the discipline treats the call itself as a write.
3. Allocate a new frame. The callee's reference parameters become fresh references attached to parameter roots $\mathrm{par}(j)$. The new frame records, for each j, the qualified node n_j in the caller—this is what return transport (below) uses.
4. Release the locks on the caller's subtrees.

Fig. 1 illustrates a conflicting pair.

Calling `foo(r1, r2)` with $r_1 = \&\mathsf{mut}x$, $r_2 = \&\mathsf{mut}x.g$:

```
root                        r1 takes exclusive lock on subtree(x).
 +-- x     <-- locked       r2 takes exclusive lock on subtree(x.g),
      +-- f                 which is a descendant of x: lock conflict
      +-- g  <-- locked     ==> runtime safety violation.
```

Fig. 1. Call-boundary subtree locking.

Returns. On return, every reference on the shadow stack must trace back to a reference parameter of the current frame. Walking the parent pointers of its qualified node should land on some $\mathrm{par}(j)$. A reference rooted at a local or a global would dangle past the frame's lifetime and is rejected. Each surviving reference is then transported into the caller: starting from the caller's qualified node for parameter j, the runtime walks the same labels and attaches a fresh reference at the resulting node. Returning two mutable references that share a subtree is caught by the same exclusive-lock pass used at calls.

Opaque effects. Some operations have bodies the runtime cannot see: native functions implemented outside the bytecode language, closure captures, and

dynamic-dispatch targets. We handle them through a small interface: each opaque operation declares, per returned reference, which input reference argument it derives from. At a call site, the runtime conjures a fresh child of that argument's node and attaches the returned reference to it; ordinary call-boundary checks still apply. The interface is part of the trusted base—an incorrect declaration silently weakens the guarantees. We return to this in Sect. 6.

5 Precision against the Trace

The rules in Sect. 4 are checked against the program's actual execution, not against a join over the program's possible executions. This is the discipline's main precision advantage over the corresponding static check, and the reason we expect a runtime model in this setting to be useful at all.

What the runtime knows that static analysis does not. The simplest illustration is a program with a conditional move:

```
let x = ...;
let r = &x.f;
if rare_condition() { move x } // invalidates subtree(x) only here
*r // safe whenever rare_condition() is false
```

A sound static analysis sees the move on one branch and has to conclude that `r` may be invalid by the time `*r` runs; it joins the two branches and reports the union. The discipline sees something more specific. By the time control reaches `*r`, `rare_condition()` has either evaluated to true or to false, and the runtime knows which. On the branch where the move did not happen, `r` is still attached to its node, no event has poisoned it, and the read succeeds. The discipline accepts a strictly larger set of executions than the static check, by construction.

Relation to static checking. We are not trying to port a static borrow checker to runtime. The rules in Sect. 4 were calibrated so that each rule poisons a reference only on cases that the corresponding static analysis would also reject when reached on every branch; this is the sense in which the discipline is meant to be at least as permissive as its static counterpart along a concrete trace. We do not prove that here. The natural proof obligation, for each static rule, is to identify a runtime invariant strong enough to rule out the same unsafe events. This is the most important open piece of the work, discussed further in Sect. 6.

Where the cost is worth it. Per-instruction shadow-state updates are not free. The discipline pays its way in contexts where static checking is unavailable or untrusted—bytecode that bypasses the verifier (fuzzing harnesses, mutated bytecode, dynamic-dispatch boundaries), and effects whose body is opaque to the bytecode language (native code). On verifier-protected execution paths, the discipline can be turned off entirely, or shifted into a trace-and-replay mode that records the relevant events during execution and checks them offline.

6 Discussion and Future Work

Formal soundness. The most important open piece is a small-step operational semantics with a soundness theorem: if a program never uses a poisoned reference, never has two overlapping mutable arguments at a call, and never returns a reference that does not trace back to a parameter, then its reference operations preserve type and memory safety. The rules in Sect. 4 are designed to make this provable.

Opaque effects. The interface for native functions, closure captures, and dynamic-dispatch targets is small but part of the trusted base. An opaque operation declares which of its reference arguments each returned reference is derived from; the runtime treats the operation as a black-box step that produces a fresh child node under that argument. This is enough to keep the discipline working at the boundary, but correctness then depends on the declared model. A natural follow-up is a unified treatment of all three cases together with the soundness condition an opaque model must satisfy.

Deployment modes. Because invalidation is a one-way bit, the discipline can be deployed in several modes. The most aggressive is *always-on*, useful as defense in depth on bytecode whose verifier outcome is not trusted. A natural relaxation is *trace-and-replay*: record the relevant events during execution and run the check on the recorded trace later, in parallel with other work. Production deployments will likely mix the two depending on the trust they place in the static verifier on each code path.

Limitations. Trusted models can silently break the guarantees if they misdescribe an opaque effect. Vector indices collapse to a single abstract label, so the discipline cannot distinguish two references into different elements of the same vector. And when a violation is detected, the runtime reports the event but does not yet emit a derivation trace explaining which earlier operation poisoned the reference; such a trace would help in debugging.

7 Related Work

Rust-style static borrow checking. The closest static relative is Rust's borrow checker, whose metatheory was given by RustBelt [3]. Both target stack-allocated state, with no first-class notion of globally-stored mutable resources, and the only dynamic borrow check in mainstream Rust is `RefCell`'s local check on a single cell. Move's reference-safety rules, in contrast, span locals, globals, and mutable cross-frame arguments; our discipline enforces them at execution time.

Per-location dynamic alias models. The closest dynamic neighbors are Stacked Borrows [2] and Tree Borrows [6]. Stacked and Tree Borrows model aliasing at memory locations: each location carries a stack or tree of borrow tags, and the

Miri interpreter checks reads and writes against this metadata to detect undefined behavior in Rust. Our model tracks references by their access paths rather than by the addresses they happen to land on, which matters because Move's reference rules include events the per-location view does not naturally express—in particular variant mutation, where a write to a discriminator invalidates references derived from the prior variant's projection. We also treat call boundaries as a first-class construct, with subtree locking and parameter-root transport doing the work that the static borrow checker does at compile time in Rust.

Capabilities, separation, runtime verification. Capability systems [4] read naturally onto our model: a node identity is the capability, and an invalidation is a revocation. Access paths play the role of a runtime footprint in the sense of separation logic [5], though we do not develop the logical reading here. The RV literature on monitoring mutable state [1] provides the broader backdrop; we specialize to one specific monitor, over the hierarchy of reference derivation. Move ships with a static reference-safety checker performing abstract interpretation over a borrow graph, which the runtime discipline is designed to interoperate with.

8 Conclusion

Move's static reference-safety rules can be enforced at execution time using a small runtime discipline. The runtime records the access paths through which each live reference was derived, and applies a single invalidation rule when destructive events disturb those paths; the rule covers writes, moves, freezes, variant mutations, calls, returns, and opaque effects uniformly, and call boundaries lock subtrees to keep mutable arguments disjoint. The discipline catches a class of unsafe uses that lifetime-based static analysis can only reject conservatively, and provides a uniform handle on opaque effects at the boundary with native code. The most important open work is a formal soundness proof against an operational semantics; the model is designed to admit one.

Disclosure of Interests. The authors have no competing interests relevant to this article.

References

1. Bartocci, E., Falcone, Y., Francalanza, A., Reger, G.: Introduction to runtime verification. In: Lectures on Runtime Verification, Lecture Notes in Computer Science, vol. 10457, pp. 1–33. Springer (2018)
2. Jung, R., Dang, H.H., Kang, J., Dreyer, D.: Stacked borrows: An aliasing model for Rust. Proc. ACM Program. Lang. **4**(POPL), 41:1–41:32 (2020)
3. Jung, R., Jourdan, J.H., Krebbers, R., Dreyer, D.: RustBelt: Securing the foundations of the Rust programming language. Proc. ACM Program. Lang. **2**(POPL), 66:1–66:34 (2018)

4. Levy, H.M.: Capability-Based Computer Systems. Digital Press (1984)
5. Reynolds, J.C.: Separation logic: A logic for shared mutable data structures. In: LICS 2002. pp. 55–74. IEEE (2002)
6. Villani, N., Dang, H.H., Jourdan, J.H., Jung, R., Dreyer, D.: Tree borrows. Proc. ACM Program. Lang. **9**(PLDI) (2025)